Can Small Defenders Triage Without Frontier Models? Open Local Models versus Hosted Decision Models for Human-on-the-Loop SOC Triage

Yaniv Zimmer[1], **Yuval Goldberg**[1], **Noam Cohen**[1]

[1]Torq
firstname.lastname@torq.io

Abstract

When a model triages every alert and analysts review only some, the right measure is not accuracy but the trade-off between missed malicious alerts and false escalations, with one miss costing as much as c false escalations. Smaller defenders without frontier LLMs can use a cheap hosted decision model or, if they cannot send alerts off-site, small open models trained on their own alerts on one GPU. We compare both with frontier LLMs on 1,492 analyst-reviewed production alerts from three tenants and on twelve organizations of the public GUIDE benchmark; hosted models get one prompt and one or more retrieved exemplars (decision models on GUIDE also 5 and 20). In the regime our analysts set before the study ($c \leq 10$), a local model was cheaper on both datasets. Under an 80% confidence gate, OpenAI's Decisions API with one exemplar became cheaper than RiskFit above about $c=14$ on production alerts and only above $c=125$ on GUIDE, and 20 exemplars narrowed the gap on GUIDE. The local advantage comes from usable confidence: at full coverage, RiskFit and the Decisions API with one exemplar cost the same on production alerts. A 0.6B open LM fine-tuned as a tiny local decision model was also cheaper than the Decisions API on both datasets. A recall-hinge loss (RiskFit) helped raising malicious recall from 12 to 46+-1 of 73 alerts.

Code — https://github.com/YanivZimmer/SocTriage

Introduction

Most alerts that SOC analysts see need no action (Alahmadi, Axon, and Martinovic 2022), so triage is an obvious task to automate. Two properties of the SOC make this harder than ordinary text classification. First, the risk is asymmetric: a malicious alert labeled benign is a missed intrusion, while a benign alert labeled malicious costs analyst time. Second, triage is *human-on-the-loop*: a model labels every alert, and analysts review a subset. The two counts that matter are the malicious alerts that are automated and missed and the false escalations that analysts must clear, weighed by what a miss costs relative to a false escalation; accuracy mixes them.

A smaller defender (defender with limited resources of efficiency concerns), without a frontier-LLM budget or permission to send alerts off its infrastructure, has several options: open or proprietary decision models, which are cheaper and provide a confidence for the verdict; a frontier or small open LLM; or a local model trained on its own alerts. The first two are easy to deploy, but we show that at moderate miss costs they cost more and hide failure modes, while on a small team's production alerts a local model trained for the task does better despite being much smaller and simpler.

The gap is largest when the model's confidence is used. Frontier-LLM deployments often ignore confidence, but a security model needs it to send the alerts it is unsure about to an analyst first. We automate the 80% of alerts on which the model is most confident and route the other 20% to analysts, whose verdicts serve incident response and become training labels for the next model.

A hosted *decision model* is a prompt-based language model that returns a class with a confidence score instead of generated text (TypeSafe AI 2026; OpenAI 2026). It needs no training, was deterministic in our runs (unlike sampled frontier-LLM output) and is easy to threshold. All our production calls, including the evasion study and the forward test, cost under $1.50. Decision models are, however, hosted and of undisclosed size, while a defender can run the text encoders and other local models we evaluate itself. Generative LLMs can return different verdicts for the same input even at temperature 0 (Atıl et al. 2025; Ouyang et al. 2023), but SOC automation needs repeatable verdicts that an analyst or auditor can replay. The local models are deterministic at inference.

A frontier LLM needs no training data but returns verbalized confidence that is poorly calibrated (Xiong et al. 2024). We ask which option a defender should choose at a given miss cost. Comparing the options on accuracy at one operating point, a known pitfall in security ML (Arp et al. 2022), hides the trade-off a SOC has to make.

We compare all three families on 1,492 analyst-reviewed alerts from three production tenants. On twelve organizations of the public GUIDE benchmark, where no legal limits apply, we add a second decision model (Jev), Claude Opus 5.5 as the frontier LLM and CyberSecQwen-4B, an open LLM a defender can run on its own hardware. Each method is scored at its default verdict, at thresholds a deployment could tune, and under a confidence gate that automates the most confident alerts and routes the rest to analysts. We also test RiskFit, a BERT encoder trained with a hinge on soft malicious recall, and a tiny local decision model fine-tuned from a 0.6B open LM. Our contributions are:

- An evaluation, on production and public data, of local

At c=10	Production	GUIDE
80% gate: RiskFit / CE	118 / 98	156 / 185
80% gate: Tiny Decision	105	150
80% gate: Decisions + ex.	154	500
80% gate: Decisions + 20 ex.	–	210
Full coverage: RiskFit	234	329
Full coverage: Decisions + ex.	233	740
Crossover c^*: RiskFit / CE	14 / 16	125 / 55

Table 1: Cost $c\cdot$misses$+$false escalations at c=10 for the two local models fixed before the study, the tiny local decision model (Qwen3-0.6B-LoRA) and the Decisions API with one or 20 retrieved exemplars (+ ex.; 20 only on GUIDE), when the 80% most confident alerts are automated and at full coverage (default verdicts), and the crossover c^* above which the Decisions API is cheaper under the gate.

ML models, hosted decision models and generative LLMs with a cost ratio that reflects the security team's workload, at default, validation-tuned and gated operating points (RQ1, RQ2).

- A tiny local decision model for the SOC: Qwen3-0.6B fine-tuned with LoRA returns a deterministic verdict with a confidence, like a hosted decision model, but on the defender's own GPU. Under the 80% gate at c=10 it was cheaper than the Decisions API with one exemplar on both datasets (105 vs. 154; 150 vs. 500 on GUIDE), lowest of any method on GUIDE after 10 training epochs (150 vs. 156 for RiskFit, not significant) and fifth among the local models on production alerts.
- A recall-hinge loss, which we call RiskFit, improved detection under natural (unbalanced) sampling: cross-entropy caught 12 of 73 malicious alerts and RiskFit 46+-1 (Holm $p<0.001$; higher in every seed). With balanced sampling it was almost equivalent to cross-entropy, because rebalancing already drives training recall to saturation, after which the hinge is inactive (RQ1).
- When a miss costs at most 10 times as much as a false escalation ($c\leq10$, our main regime), a local model beats the decision models on both datasets under the 80% gate, and the frontier LLMs where tested, so defenders should train their own models. Above $c\approx14$ for RiskFit, the Decisions API with one exemplar becomes cheaper on production alerts (Table 1).
- Weak labels from the production LLM do not replace analyst labels: at review budgets of 25% and above, the revealed analyst labels alone give the better model on κ, and on cost unless a miss costs more than about 20 (CE) or 27 (RiskFit) false escalations (RQ3).
- A public, reproducible GUIDE replication and an audit of what the private benchmark can show.

The work addresses tools and metrics for AI in cybersecurity (the cost-ratio evaluation), decision-making under uncertainty (the confidence gate) and defending infrastructure without frontier LLMs: every local model is built from open components and trains on a single GPU or CPU.

Background and Threat Model

Triage verdicts. Each alert gets one of three verdicts: *false positive* (FP; the detection fired incorrectly), *true positive, benign* (TP-benign; a correct detection of authorized activity) or *true positive, malicious* (TP-malicious). The FP/benign boundary depends on tenant policy and does not matter for security, so malicious-versus-rest is our primary task and three-way agreement is secondary.

Threat model and cost. The asset is the tenant's infrastructure. The failure we guard against is a *miss*: a malicious alert that receives a non-malicious verdict without human review. Avoiding misses costs false escalations that analysts must clear. With a cost ratio c, one miss costs as much as c false escalations, so a configuration costs $c\cdot$ misses $+$ false escalations. We take $c\leq10$ as the main regime. It is our SOC analysts' rule of thumb, set before the study by weighing the follow-up work a missed malicious alert causes against the time an analyst spends processing an alert, which varies by alert type; since c differs between teams, we report the crossover c^* for every comparison (Table 1, Figure 2). For many defenders data egress is a hard constraint: regulated, air-gapped or contract-bound teams cannot send alerts to a third-party model, so only local models are open to them. The attacker controls alert content (file names, command lines, indicators) but not the detector, the triage model or analysts' labels; we test simple, non-adaptive evasions (RQ2) and leave adaptive attackers, poisoned feedback and prompt injection out of scope (Limitations).

Methods Compared

Local model. Each alert is serialized to text from 21 fields (e.g., alert name, severity, observables with threat-intelligence enrichment, process chain, command and vendor response), truncated at 1,024 tokens; no label, analyst comment, status or LLM verdict is included. Following FastFit (Yehudai and Bandel 2024), the text is encoded by SecureBERT 2.0 (Aghaei et al. 2025), a ModernBERT-based security encoder of about 150M parameters, fine-tuned per tenant with a LoRA adapter (Hu et al. 2022) (rank 32, α=64, dropout 0.1, attention projections) and a three-way head.

RiskFit objective. Let p_i be the predicted probability that alert i is malicious and $\mathcal{M}$ the malicious alerts in a micro-batch. With soft recall $R_{\text{soft}}=\frac{1}{|\mathcal{M}|}\sum_{i\in\mathcal{M}}p_i$, RiskFit minimizes

$$\mathcal{L}=\mathcal{L}_{\text{CE}}+\gamma\,\mathcal{L}_{\text{SupCon}}+\lambda\max(0,\rho-R_{\text{soft}}),\qquad(1)$$

where $\mathcal{L}_{\text{SupCon}}$ is the supervised contrastive loss (Khosla et al. 2020). The hinge pushes only while soft malicious recall is below the target ρ. Micro-batches of four are sampled 50/50 malicious/non-malicious (gradients accumulated over nine), which already pushes training recall up; RQ1 tests what the hinge adds with and without this rebalancing. We use ρ=0.9, λ=2, γ=0.5, set by convention rather than tuned (Limitations), and also report ρ=0.8 and λ=4. With λ=0 the objective is our cross-entropy (CE) baseline, which shares the backbone, balanced sampling and SupCon term. CE denotes the benchmark run on a T4 GPU. For paired comparisons

with RiskFit we rerun CE on RiskFit's A100 (*CE-A100*); on two tenant-folds, RiskFit's code with λ=0 and the CE code gave identical predictions on the same hardware. The code also supports a soft false-positive penalty β, set to 0 throughout as fixed before any result; with balanced sampling, β=0.25 cut false escalations by 16 $\pm$ 13 per seed without changing any conclusion.

Baselines. *Local models:* the same backbone with CE or focal-Tversky loss (Lin et al. 2017; Abraham and Khan 2019); MaxSim, FastFit's similarity classifier; k-nearest neighbors (k=5) over frozen SecureBERT embeddings, a stand-in for a semantic cache of past verdicts; LightGBM (Ke et al. 2017) on structured fields; SetFit with LoRA (Tunstall et al. 2022); and a *tiny local decision model*: Qwen3-0.6B (Yang et al. 2025) fine-tuned with LoRA as a sequence classifier with a trained score head, which returns class probabilities rather than text and is deterministic at inference (up to 1.7 A100-hours per training fold, against about 13 minutes for an encoder). *Frontier LLM:* on production alerts, Gemini 2.5 Pro (Comanici et al. 2025), the model behind the production verdicts, run with our study prompt (temperature 0, provider-default thinking), returning a verdict and a self-reported confidence. It runs zero-shot or with one exemplar: the most similar training alert and its analyst label, retrieved label-blind on alert name, executables, MITRE techniques and observable types. *Hosted decision models:* OpenAI's Decisions API (OpenAI 2026) (`gpt-6-luna`) gets the same inputs and exemplars and answers a three-way *choice* question and a *predicate* question for maliciousness; it was deterministic across 3,300 repeated calls. On GUIDE, where data-egress restrictions do not apply, we add TypeSafe's Jev (TypeSafe AI 2026) (`jev-1.14.0`, 2026-11-01) with byte-identical inputs, the same choice question and the predicate as its yes/no question. As the frontier LLM on GUIDE we use Claude Opus 5.5 (Anthropic 2026) (`claude-opus-5-5`, medium effort, 2026-11-03), with the same inputs and instructions and no tools, run once per prompt through the Claude Code command-line interface, returning a verdict, a self-reported confidence and p(malicious). For both decision models we also retrieve the five and the twenty most similar training prompts, in the same label-blind way. As an open model a small defender could run on its own hardware, we use CyberSecQwen-4B (Mulia 2026) (`athena129/CyberSecQwen-4B`, a fine-tune of Qwen3-4B-Instruct-2507 for mapping CVEs to CWE categories and threat-intelligence questions, not alert triage), zero-shot with the same instructions, and its base model Qwen3-4B-Instruct as a control; its p(malicious) is the softmax over the log-probabilities of the three verdict labels (scoring details are in the code release).

Operating points and gate. For models with class probabilities we report the default verdict (argmax) and a threshold on p(malicious) tuned on the fold's validation split to a target false-positive rate, as a deployment could. The gate automates the alerts with the highest confidence (maximum class probability, or the LLM's or Decisions API's own confidence) and routes the rest to an analyst. We set the cutoff in two ways and report both. The *test-quantile gate*, our default, automates the top 80% of each test fold; it compares methods at matched coverage but could not be set in deployment. The *deployable gate* uses the threshold that automates 80% of the validation split. An automated malicious alert not labeled malicious is a *miss*; routed alerts are assumed to be handled correctly and are not charged.

Weak supervision from LLM verdicts. In production modern SOC systems it is common that an LLM labels every alert and analysts review a subset. We simulate a review budget k by revealing the analyst label for a random k% of each training pool and keeping the production LLM's verdict for the rest (RiskFit and CE weight revealed labels 2:1, as in the benchmark), and compare with training on the revealed labels alone. We test $k \in \{0, 1, 2, 5, 10, 25, 50\}$% ($k$=0 is the LLM's verdicts alone), with one random draw and two more at 1–5%.

Experimental Setup

Research questions. **RQ1:** How do default and validation-tuned operating points compare, and when does a recall-oriented loss help? **RQ2:** Under a confidence gate, which methods best trade unreviewed misses against false escalations, especially at $c \leq 10$? **RQ3:** Does training on LLM verdicts plus some revealed analyst labels beat training on those labels alone?

Public benchmark (GUIDE). We report the public benchmark first, because every result on it is reproducible. We use twelve organizations of the public GUIDE incident-triage benchmark (Freitas et al. 2025), chosen in a pre-registered hash order (with documented amendments) among organizations with at least 300 distinct prompts and 10 malicious ones. Many incidents render to identical text, so the unit is a distinct prompt labeled by its incidents' majority grade (TruePositive, BenignPositive, FalsePositive), split 70/15/15 per organization; the test sets hold 1,079 prompts, 456 malicious. RiskFit, CE, LightGBM, MaxSim, kNN and the tiny decision model are trained per organization. Both decision models and Claude Opus 5.5 run zero-shot and with one exemplar retrieved label-blind from the organization's training split. In addition we tested the decision models also with five and twenty, and CyberSecQwen-4B and its base model zero-shot. Gemini was not run on GUIDE as a stronger Opus 5.5 model was used as the Frontier LM. GUIDE labels are independent of our production LLM.

Production alerts. We use 1,492 alerts from three anonymized production tenants, A, B and C (412, 707 and 373 alerts, of which 17, 39 and 17 malicious), collected between September 2025 and September 2026. Analysts chose which alerts to review, and we assume this choice was random with respect to the true verdict (it may not be on tenant B; Limitations). Every label is an analyst verdict on an alert the production LLM (Gemini 2.5 Pro with the production system's own prompt; our Gemini runs use the same model with our study prompt) had already triaged: 528 labels are *corrections* of the LLM's verdict, only six of them malicious, and 964 were reviewed without correction (943 of them equal the LLM's export-time verdict). All methods share three folds,

stratified by label with a fixed seed; each training fold holds out 15% for validation.

Recall-hinge experiments. Two production experiments test the hinge. A pre-registered test compares RiskFit with CE-A100 over seven seeds and trains RiskFit with $\rho \in \{0.7, 0.8, 0.9, 0.95\}$ (three seeds each) to check whether ρ sets test recall. A follow-up, designed after that test and pre-registered before training, crosses sampling (balanced, as in our recipe, or natural, at the data's class ratio) with four losses (CE, and hinges with ρ/λ of 0.9/2, 0.95/5 and 0.99/8) over three seeds; with natural sampling most micro-batches hold no malicious alert, so there the hinge uses soft recall over the 36-alert accumulation window. Tests are paired Wilcoxon tests over tenant-fold-seed units with Holm correction.

Metrics and statistics. We report malicious alerts caught and false escalations pooled over folds, Cohen's κ (Cohen 1960) as a secondary three-way measure, ROC- and PR-AUC for $p(\text{malicious})$, and gate metrics. The tenant (or GUIDE organization) is the unit of replication, and paired tests across organizations use Holm correction (Holm 1979). Gate bootstraps resample alerts within tenant and fold (2,000 replicates); a second bootstrap (another seed, intervals within one miss of the first) gives the per-model and evasion comparisons. RiskFit trains for up to 50 epochs (learning rate 10^{-4}, early stopping on validation accuracy) on one A100, about 13 minutes per run.

Results

Public Benchmark (GUIDE)

Confidence gating. With the 80% most confident prompts automated (Table 2), the tiny decision model had the lowest cost at c=5 and 10 (150 at c=10; RiskFit 156, CE 185) and RiskFit at c=20; with one seed they are not separable (cheapest in 64%, 30% and 6% of bootstrap replicates at c=10, without the twenty-exemplar conditions; Holm $p\approx1$ between them). Including all 18 conditions, a local model was cheapest in 97.6% of replicates at c=10 and 86.8% at c=20. RiskFit and CE dominated Opus with an exemplar, with fewer misses and fewer false escalations, in 84% and 60% of replicates. The Decisions API with one exemplar missed about as many malicious prompts as RiskFit (8 against 11; 95% intervals 3–15 and 5–19) but made nine times as many false escalations, more than RiskFit and CE in all twelve organizations (Holm p=0.003); RiskFit stayed cheaper up to $c\approx125$ (95% interval 30 to unbounded). With twenty exemplars it fell to 12 misses and 90 false escalations (cost 210): RiskFit and the tiny decision model dominated it at the point estimate, in 58% and 52% of replicates, and CE crossed it at $c\approx18$. CyberSecQwen-4B flags almost every prompt, so RiskFit is cheaper only below $c\approx41$; its base model ranked at chance too (ROC-AUC .45), so the security fine-tune shifted verdicts toward malicious without adding a triage signal. Against one-exemplar conditions, every crossover exceeds c=10 and is higher than on production alerts (Figure 2).

Default and tuned operating points. At the default verdict (Figure 1), RiskFit caught 431 of 456 malicious prompts with 79 false escalations among 623, CE 429 with 70, the tiny decision model 423 with 50, kNN 415 with 53, LightGBM 395 with 65 and MaxSim 380 with 35. The hosted models caught 399 to 452, with 126 to 620 false escalations, and the locally run CyberSecQwen-4B labeled 1,077 of 1,079 prompts malicious. RiskFit's recall beat LightGBM's in ten of twelve organizations (mean +0.15, Holm p=0.024) and matched CE's (+0.01). Thresholds tuned on validation to RiskFit's false-positive rate did not help the hosted models: they lost far more catches than false escalations (Holm p=0.008), and at c=10 the decision models cost 1,381 and 1,173 with five exemplars and 541 and 875 with twenty, against 325 for the tiny decision model, 380 for RiskFit and 451 for CE. With up to five exemplars every hosted configuration ranked malicious prompts worse than every local model; with twenty, the Decisions API (ROC-AUC .86) matched them (Table 2). The zero-shot models ranked at or below chance.

Method	Miss	FE	c^* vs. RiskFit	c^* vs. CE	AUC
RiskFit	11	46	–	–	.85±.14
CE	15	35	–	–	.88±.10
LightGBM	31	39	–	–	.85±.10
MaxSim	38	17	–	–	.86±.14
kNN	21	28	–	–	.85±.08
Tiny Decision	12	30	–	–	.88±.11
Decisions, 0-shot	0	497	41	31	.49±.17
Decisions + ex.	8	420	125	55	.66±.21
Decisions + 5 ex.	24	109	∞	∞	.76±.21
Decisions + 20 ex.	12	90	∞	18	.86±.10
Jev, 0-shot	4	492	64	42	.45±.13
Jev + ex.	27	381	∞	∞	.60±.24
Jev + 5 ex.	38	245	∞	∞	.72±.18
Jev + 20 ex.	29	213	∞	∞	.81±.14
Opus, 0-shot	5	490	74	46	.47±.12
Opus + ex.	16	241	∞	∞	.64±.22
CyberSecQwen	0	502	41	31	.45±.16
Qwen3-4B base	59	437	∞	∞	.45±.10

Table 2: GUIDE, 80% test-quantile gate: misses (of 456) and false escalations (FE, of 623; 0-shot: zero-shot), so cost is $c \cdot \text{Miss} + \text{FE}$; the crossover c^* below which RiskFit or CE is cheaper (∞: at every c), and malicious ROC-AUC (mean $\pm$ std over the twelve organizations; hosted models: predicate probability). One seed per local model; Tiny Decision (Qwen3-0.6B-LoRA) trained for 10 of 20 epochs; kNN's misses range from 19 to 21 with tie order at the cutoff.

Exemplars. The retrieved exemplar's label was correct for 86% of prompts, yet it moved the decision models little: the Decisions API and Jev agreed with it on 55% and 51% of prompts, against 45% and 43% zero-shot (Opus: 79%, from 45%). With five exemplars they followed the most similar one 85% and 68% of the time, and with twenty 89% and 71%, cutting false escalations at the gate by 79% and 44% from one exemplar. They then approach a model-free vote over labeled neighbors, which caught 410 with 75 false escalations at full coverage, costing more than RiskFit and CE at c=10 (535

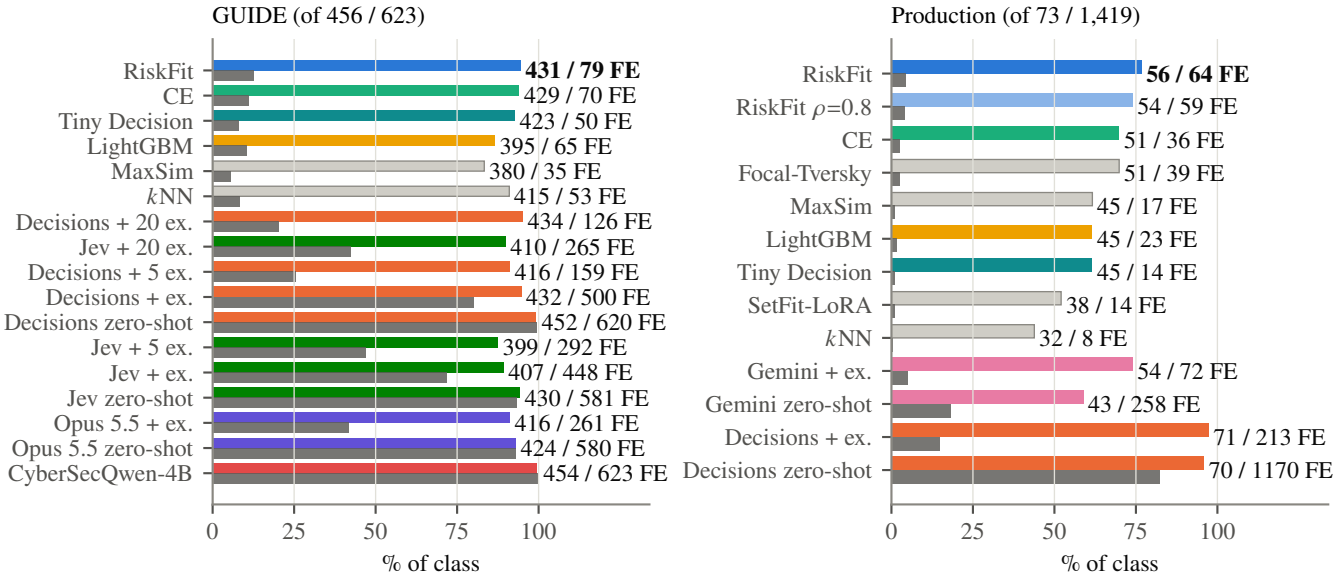


Figure 1: Default (argmax) verdicts at full coverage on GUIDE (left) and production alerts (right): malicious alerts caught (colored bars; of 456 and 73) and non-malicious alerts falsely escalated (gray bars; of 623 and 1,419). CE: benchmark T4 run (CE-A100: 50 with 28). One seed per model.

against 329 and 340).

Production Alerts

Default and tuned operating points. Over seven seeds, RiskFit caught 50.0 ± 3.2 of 73 malicious alerts with 39 ± 13 false escalations among 1,419, and CE-A100 49.4 ± 2.4 with 30 ± 7: a difference of +0.6 catches (95% CI −1.1 to +2.3) and +9 false escalations (CI +4 to +15), at equal cost under every gate (Holm p=1.0). Seed 42, shown in Figure 1, was RiskFit's most favorable. The Decisions API caught more (71 with an exemplar, 70 zero-shot) with 213 and 1,170 false escalations, and Gemini fewer. RiskFit did not rank better: malicious ROC-AUC on tenants A, B and C was 0.956, 0.905 and 0.993, against CE's 0.961, 0.926 and 0.992. With thresholds tuned on validation to RiskFit's false-positive rate, kNN caught 55 with 48 false escalations, RiskFit 56 with 93 and the Decisions API with an exemplar 54 with 73.

When the recall hinge helps. Training RiskFit with recall targets ρ of 0.7, 0.8, 0.9 and 0.95 gave test recall of 0.72, 0.74, 0.73 and 0.74 (Spearman 0.008), so ρ does not set the operating point; in the pre-registered test, RiskFit was no closer to its target than CE thresholded on validation (Holm p=1.0). With balanced sampling, as in our recipe (Table 3), CE alone drove training soft recall from 0.36 to 0.99; the hinge was active on every step of the first epoch but on 3%, 15% and 5% of steps by the last (for the three strengths), and no hinge strength differed from CE (Holm $p \geq 0.17$). With natural sampling, CE caught 11.7 of 73 malicious alerts, its training soft recall ending at 0.46, while every hinge raised it to 45–47 (Holm $p<0.001$; higher in every seed) at about 28 more false escalations, matching balanced CE. The sampling-by-loss interaction is significant for catches and full-coverage cost (Holm $p<0.001$) and for gate cost (Holm $p \leq 0.032$). The hinge thus substitutes for class rebalancing.

Sampling	Loss	Caught	FE	Gate
Balanced	CE	47.3±2.5	27.0±6.6	137±39
	$\rho.9/\lambda 2$	49.3±3.5	31.3±4.7	137±25
	$\rho.95/\lambda 5$	51.3±3.1	36.3±7.5	101±11
	$\rho.99/\lambda 8$	46.7±1.2	33.0±4.4	120±23
Natural	CE	11.7±0.6	1.7±1.5	227±45
	$\rho.9/\lambda 2$	46.0±4.0	30.3±13.1	122±33
	$\rho.95/\lambda 5$	45.3±3.8	31.3±10.2	94±4
	$\rho.99/\lambda 8$	47.0±3.6	28.7±11.0	83±28

Table 3: Recall hinge by sampling on production alerts: malicious caught (of 73), false escalations (FE, of 1,419) and cost $10 \cdot$ misses + FE at the 80% gate (Gate), mean ± std over seeds 0–2 (9 tenant-folds each). Natural: the data's class ratio, with the hinge over the 36-alert accumulation window.

Confidence gating. Table 4 shows unreviewed misses and false escalations when the 80% most confident alerts of each test fold are automated. Four configurations are Pareto-optimal: the Decisions API with an exemplar (0 misses, 154 false escalations), RiskFit ρ=0.8 (6, 25), MaxSim (7, 4) and kNN (16, 2). At c=10, MaxSim (cost 74), RiskFit ρ=0.8 (85), CE (98) and RiskFit (118) each cost less than every hosted configuration, the cheapest of which were Gemini with an exemplar (151) and the Decisions API with an exemplar (154). With the cheapest of the ten local configurations chosen after seeing test results, a local model was cheapest in 99.98% of 2,000 bootstrap replicates; MaxSim and RiskFit

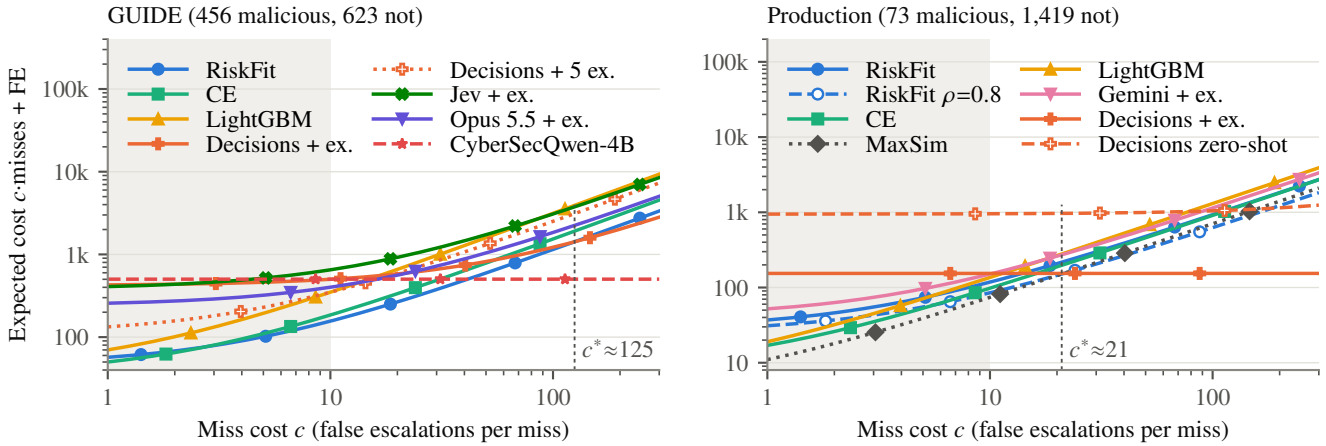


Figure 2: Cost curves (Drummond and Holte 2006): expected cost $c \cdot$ misses + false escalations at the 80% test-quantile gate (log axes; shaded: the main regime $c \leq 10$; dotted: the best local model's crossover with the Decisions API with an exemplar). On production alerts that crossover is 14 for RiskFit, 16 for CE and 11 for LightGBM, and against the zero-shot Decisions API 115, 117 and 78. GUIDE crossovers are in Table 2; reweighted to the production class ratio, RiskFit's and CE's rise to 896 and 695.

Method	Miss	FE	Method	Miss	FE
Decisions + ex.	**0**	154	CE	9	8
RiskFit ρ=0.8	**6**	**25**	Tiny Decision	10	5
MaxSim	**7**	**4**	Gemini + ex.	11	41
Focal-Tversky	7	18	LightGBM	13	6
RiskFit λ=4	8	31	kNN (frozen)	**16**	**2**
RiskFit	9	28	SetFit-LoRA	16	4

Table 4: Gate at 80% coverage: unreviewed malicious misses (of 73) and false escalations (FE, of 1,419), pooled over tenants. + ex.: with one retrieved exemplar. Bold: the four configurations not dominated on both counts. Zero-shot Gemini (21 misses, 181 false escalations) and the zero-shot Decisions API (1 miss, 945) are omitted.

ρ=0.8 cannot be ordered (95% intervals of 2–14 and 2–13 misses). We compared the six local models of the evasion study (Table 5) one-to-one with the Decisions API with an exemplar: at c=10, RiskFit, RiskFit ρ=0.8, CE and MaxSim were cheaper in 89%, 95.5%, 93% and 99% of replicates, kNN in 42% and LightGBM in 67%.

Seeds, deployable gate and crossover. Under the deployable gate, every seed of RiskFit and CE (seven seeds) and MaxSim (three) still cost less than the Decisions API with an exemplar at c=10 (mean $\pm$ std 104 $\pm$ 27, 101 $\pm$ 20 and 90 $\pm$ 46, against 155). Most of the local advantage comes from gating on the local models' confidence: among default verdicts at full coverage, the Decisions API with an exemplar was already the cheapest at c=10 (233 against RiskFit's 234), although kNN thresholded on validation to a 5% false-positive rate cost 204. Under the deployable gate, RiskFit (6 misses, 26 false escalations) cost less than the Decisions API with an exemplar (0, 155) in 98% of replicates at c=10, but it automated only 68%, 75% and 82% of tenants A, B and C's alerts, and our cost does not charge for routed alerts. The Decisions API with an exemplar becomes cheaper than RiskFit above $c \approx 14$ and than CE above 16 (Table 1); against the best local configuration, chosen after seeing test results, the crossover is about 21 (RiskFit ρ=0.8 21.5, MaxSim 21.4; 95% interval for MaxSim 11–71; Figure 2). Gemini with an exemplar (11, 41) is dominated by RiskFit and MaxSim. Its verbalized confidence took at most six distinct values per fold, and its gate matched CE on A, was close on C and failed on B (9 misses, 36 false escalations).

Later alerts. Tenant A's analysts reviewed 433 later alerts (11 malicious), scored by models trained on, or with exemplars from, the earlier ones. At full coverage RiskFit caught 9 with 8 false escalations, CE 9 with 50 and LightGBM 5 with 2. At matched 80% automation, RiskFit and LightGBM had 2 unreviewed misses with 1 and 0 false escalations, CE 2 with 28, and the Decisions API with an exemplar 1 with 56, so RiskFit and LightGBM cost less than it up to $c \approx 55$; on tenant A's benchmark alerts RiskFit had no such advantage at the gate (one more miss, equal false escalations). The Decisions API's false-escalation rate with an exemplar rose from 0.045 to 0.18 (RiskFit: 0.019), as 66% of the later alerts had no exemplar above the similarity threshold (before: 38%). Validation gate thresholds carried forward poorly: instead of 80% they automated 15% (CE), 20% (RiskFit), 39% (Decisions API with an exemplar; 0 misses, 20 false escalations), 72% (LightGBM) and 93% (zero-shot). With 11 malicious alerts, these miss differences are only indicative.

Evasive alerts. We perturbed the attacker-controllable fields of the 73 malicious production test alerts with fixed, non-adaptive rules, keeping each gate's unperturbed threshold (Table 5). P1 renames attacker binaries to benign system names, P2 obfuscates command lines (encoded PowerShell, caret insertion), P3 gives indicators with threat-intelligence hits fresh values and strips their enrichment, and P4 com-

Method	None	P1	P2	P3	P4	R0
Decisions + ex.	0	0	0	1	1	2
RiskFit ρ=0.8	6	6	6	7	7	8
MaxSim	7	7	7	8	8	7
RiskFit	9	9	9	9	9	8
CE	9	9	9	11	11	10
LightGBM	13	13	13	**23**	**23**	13
kNN (frozen)	16	17	17	14	16	18

Table 5: Unreviewed misses (of 73) at the 80% gate with the unperturbed threshold, with perturbed malicious alerts (P1–P4: see text; R0: vendor Response field removed). False escalations are unchanged. Bold: paired 95% interval of the change excludes zero.

bines all three; they changed 29, 24, 30 and 47 alerts. The local encoder models gained at most two misses (every paired 95% interval includes zero), and the Pareto set and the production crossover ($c\approx21$) did not change. LightGBM, whose features include enrichment, missed 10 more alerts under fresh infrastructure (95% interval 4–17). The Decisions API with an exemplar missed one alert under P3 and P4, and two when the vendor Response field was dropped (R0, a leakage control: this field alone identifies 7 of tenant A's 17 malicious alerts).

Weak supervision from LLM verdicts. The production LLM's verdicts over-called malicious at every budget (Table 6): at 25%, every weak-label model escalated several times more non-malicious alerts than with all labels, and agreement stayed low (κ .27 for RiskFit and .30 for CE, against .59 and .60 with all labels). At $k\leq5\%$ the revealed labels alone were rarely enough to train on, so weak labels won on κ there. At 25–50%, the revealed labels alone won for RiskFit, CE, LightGBM and SetFit-LoRA; weak labels caught more malicious alerts but with far more false escalations (RiskFit at 25% on matched folds: 57 of 67 with 546 false, against 39 of 67 with 55). On tenant B the LLM labeled about 330 of 471 training alerts malicious against 26 true ones. RQ3 revealed labels at random; in deployment the gate routes the least-confident 20% of alerts to analysts, whose verdicts become the next training labels, which is uncertainty sampling in active learning (Lewis and Gale 1994). The next model should learn from these verdicts, not the LLM's; RQ3 does not measure how much uncertainty-selected labels add over random ones.

Limitations

Data and evaluation. The production benchmark has 73 malicious alerts (about six per fold on tenants A and C) and random rather than time-ordered folds, which can overstate performance on evolving security data (Sommer and Paxson 2010; Pendlebury et al. 2019); 33.5% of tenant A's test alerts have an exact duplicate in training, and 57.5% of A's and 21% of B's have a training neighbor at cosine $\geq$0.99. Labels are analyst verdicts given after the production LLM's verdict, so the benchmark rewards agreement with that LLM, and on tenant A the vendor Response field and threat-intelligence

Method	Weak+25%	25% only	Full
RiskFit	61/556/.27	39[a]/55/.43	56/64/.59
CE-A100	59/444/.30	43[a]/181/.34	50/28/.60
LightGBM	54/363/.34	25[b]/49/.45	45/23/.63
SetFit-LoRA	50/486/.26	21/21/.48	38/14/.60
kNN	51/407/.32	11/13/.28	32/8/.60

Table 6: Weak supervision with 25% of analyst labels revealed: malicious caught / false escalations / mean κ, pooled over 9 tenant-folds (73 / 1,419 unless noted). *Weak*: LLM verdicts with 25% of analyst labels revealed; *25% only*: those analyst labels alone; *Full*: all analyst labels. [a]One fold failed by construction (no malicious training alert): 67 malicious and 1,301 non-malicious evaluated. [b]Two folds failed (a validation class absent from training): 61 and 1,170. SetFit-LoRA's 25% fold C2 had no malicious training alert and is counted with recall 0.

enrichment may carry outcome information. Tenant C's later alerts came from a different export, with severity missing from most alerts and a different non-malicious labeling convention, so we do not use them as a forward test; on them the text encoders over-escalated (RiskFit: 295 of 1,034), and we release the aggregate results. On the 280 test alerts without a training neighbor above cosine 0.95, the best local configuration changes (CE and focal-Tversky dominate), but local models still cost less than the Decisions API with an exemplar at c=10.

Where hosted models are competitive. With an exemplar, the Decisions API ranked malicious alerts on tenants A and B better than every local model. The hosted results rest on one prompt design per model and one run of Opus, whose tied confidences make its gate misses depend on tie order (16 here, 16.5 in expectation). From the zero-shot models' at-or-below-chance ranking on GUIDE (RQ1) we hypothesize that the organizations' grading conventions, rather than generic signs of maliciousness, decide GUIDE labels, so GUIDE favors methods that learn from each organization's labels. Twenty exemplars narrowed the gap (Decisions API gate cost 210 against 156 for RiskFit), and heavier retrieval may narrow it further.

Weak supervision. RQ3 is judged mainly on κ with randomly revealed labels, which do not model the uncertainty-selected labels of a deployed gate.

Adversaries and scope. The evasion rules are fixed and non-adaptive, keep the alert name (which still carries the indicator for 15 of the 30 alerts that P3 changed) and assume the detection still fires; attackers who optimize alerts toward confident benign verdicts, insiders who poison analyst corrections, and prompt injection against the hosted LLMs remain untested. The local option needs a few hundred labeled alerts per tenant, periodic retraining and gate recalibration, which we do not price.

Related Work

ML for SOC triage. GUIDE is a public incident-triage benchmark (Freitas et al. 2025); we measure several pitfalls of Arp et al. (2022). LLMs are now used across SOC tasks (Habibzadeh, Feyzi, and Ebrahimi Atani 2026; Singh et al. 2026); CORTEX (Wei et al. 2025) and AACT (Turcotte, Labrèche, and Paquette 2025) automate triage with LLM agents and from analysts' past actions, and DeepCASE routes the events it is unsure about to operators (van Ede et al. 2022). AACT, the closest system, deploys supervised local triage; we add a cost-ratio comparison against hosted decision models and LLMs.

Imbalanced and cost-aware objectives. Rebalancing training data is equivalent to moving the decision threshold (Elkan 2001), which is why our hinge is redundant with balanced sampling. Focal and focal-Tversky losses reweight hard or minority examples (Lin et al. 2017; Abraham and Khan 2019), and constrained objectives target recall directly (Eban et al. 2017); RiskFit pairs supervised contrastive learning (Khosla et al. 2020) with a simpler hinge on soft malicious recall. Weak supervision trains on noisy labels from many sources (Ratner et al. 2017); RQ3 tests the single-source case in which the noisy labeler is the production LLM.

Selective prediction and deferral. Reject options (Chow 1970), selective classification (Geifman and El-Yaniv 2017), and learning to defer (Mozannar and Sontag 2020) route uncertain inputs to an expert or stronger model and need calibrated, error-ranking confidence, which neural networks (Guo et al. 2017) and verbalized LLM confidence (Xiong et al. 2024) often lack.

Conclusion

We compared methods for human-on-the-loop triage by unreviewed misses and false escalations at a stated cost ratio c, and the answer depends on c. Below the crossover, which on production alerts was about 14 for RiskFit and 16 for CE and on GUIDE 125 and 55, a local model trained on the defender's own alerts was cheaper than every hosted configuration with one retrieved exemplar, held up under simple evasions and keeps alerts in-house; this includes our main regime $c \leq 10$ on both datasets. Above the crossover, the Decisions API with an exemplar was cheaper on production alerts, and twenty exemplars narrowed the gap on GUIDE. A tiny local decision model, a 0.6B LM fine-tuned as a classifier, matched the best encoders while keeping verdicts deterministic and in-house. A recall hinge helps only when the training data is not rebalanced; with balanced sampling it adds little. The hosted options also hid three failure modes: they over-escalated at their default verdicts, the decision models followed the exemplar's label only about half the time, and Gemini's verbalized confidence gated well on only two of three tenants. The next model should learn from the analysts' verdicts on the alerts the gate routes, in an active-learning loop; training on the LLM's verdicts gave worse models at random review budgets of 25% and above. These results rest on 73 malicious production alerts, labels anchored in the production LLM's verdicts, and single seeds on GUIDE.

Ethical Statement

Alerts were reviewed by analysts in normal SOC operations; no experiment involved human participants. The data owners permitted publication of aggregate results only; every public-data comparison is reproducible from GUIDE. Tenants are anonymized, and only Google and OpenAI processed their alerts, with the data owner's approval; TypeSafe and Anthropic saw only public GUIDE data, and the open LLMs ran locally. We release aggregate metrics and code that reproduces the GUIDE comparisons, with the hosted models' per-prompt GUIDE outputs, but no alert text, customer identifiers, per-record predictions or trained adapters. Automated verdicts can hide real attacks: we recommend keeping routed alerts under analyst review, auditing automated alerts, and checking unreviewed misses on held-out data before raising coverage.

References

Abraham, N.; and Khan, N. M. 2019. A Novel Focal Tversky Loss Function with Improved Attention U-Net for Lesion Segmentation. In *IEEE ISBI 2019*, 683–687.

Aghaei, E.; Jain, S.; Arun, P.; and Sambamoorthy, A. 2025. SecureBERT 2.0: Advanced Language Model for Cybersecurity Intelligence. In *2025 Annual Computer Security Applications Conference Workshops (ACSAC Workshops)*, 382–388.

Alahmadi, B. A.; Axon, L.; and Martinovic, I. 2022. 99% False Positives: A Qualitative Study of SOC Analysts' Perspectives on Security Alarms. In *USENIX Security 2022*, 2783–2800.

Anthropic. 2026. Claude Opus 5.5 System Card. https://www.anthropic.com/claude-opus-5-5-system-card.

Arp, D.; Quiring, E.; Pendlebury, F.; et al. 2022. Dos and Don'ts of Machine Learning in Computer Security. In *USENIX Security 2022*, 3971–3988.

Atıl, B.; Aykent, S.; Chittams, A.; et al. 2025. Non-Determinism of "Deterministic" LLM System Settings in Hosted Environments. In *Eval4NLP Workshop*, 135–148.

Chow, C. K. 1970. On Optimum Recognition Error and Reject Tradeoff. *IEEE Transactions on Information Theory*, 16(1): 41–46.

Cohen, J. 1960. A Coefficient of Agreement for Nominal Scales. *Educational and Psychological Measurement*, 20(1): 37–46.

Comanici, G.; Bieber, E.; Schaekermann, M.; Pasupat, I.; Sachdeva, N.; Dhillon, I.; et al. 2025. Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities. arXiv:2507.06261.

Drummond, C.; and Holte, R. C. 2006. Cost Curves: An Improved Method for Visualizing Classifier Performance. *Machine Learning*, 65(1): 95–130.

Eban, E.; Schain, M.; Mackey, A.; Gordon, A.; Rifkin, R.; and Elidan, G. 2017. Scalable Learning of Non-Decomposable Objectives. In *AISTATS*, volume 54 of *Proceedings of Machine Learning Research*, 832–840.

Elkan, C. 2001. The Foundations of Cost-Sensitive Learning. In *IJCAI*, 973–978.

Freitas, S.; Kalajdjieski, J.; Gharib, A.; and McCann, R. 2025. AI-Driven Guided Response for Security Operation Centers with Microsoft Copilot for Security. In *WWW Companion '25*, 191–200.

Geifman, Y.; and El-Yaniv, R. 2017. Selective Classification for Deep Neural Networks. In *NeurIPS*, volume 30.

Guo, C.; Pleiss, G.; Sun, Y.; and Weinberger, K. Q. 2017. On Calibration of Modern Neural Networks. In *ICML*, 1321–1330.

Habibzadeh, A.; Feyzi, F.; and Ebrahimi Atani, R. 2026. Large Language Models for Security Operations Centers: A Comprehensive Survey. *Journal of Electrical and Computer Engineering*, 2026(1): 3383674.

Holm, S. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics*, 6: 65–70.

Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; and Chen, W. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *ICLR*.

Ke, G.; Meng, Q.; Finley, T.; Wang, T.; Chen, W.; Ma, W.; Ye, Q.; and Liu, T.-Y. 2017. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *NeurIPS*, volume 30.

Khosla, P.; Teterwak, P.; Wang, C.; et al. 2020. Supervised Contrastive Learning. In *NeurIPS*, volume 33, 18661–18673.

Lewis, D. D.; and Gale, W. A. 1994. A Sequential Algorithm for Training Text Classifiers. In *Proceedings of SIGIR '94*, 3–12.

Lin, T.-Y.; Goyal, P.; Girshick, R.; He, K.; and Dollár, P. 2017. Focal Loss for Dense Object Detection. In *IEEE ICCV*, 2999–3007.

Mozannar, H.; and Sontag, D. 2020. Consistent Estimators for Learning to Defer to an Expert. In *ICML*, 7076–7087.

Mulia, S. 2026. CyberSecQwen-4B: A Compact CTI Specialist Fine-Tuned from Qwen3-4B-Instruct-2507. https://huggingface.co/athena129/CyberSecQwen-4B. Revision `6c82bf13`, Apache-2.0, accessed 2026-11-03.

OpenAI. 2026. Decisions API Guide. https://developers.openai.com/api/docs/guides/decisions. Accessed: 2026-10-30.

Ouyang, S.; Zhang, J. M.; Harman, M.; and Wang, M. 2023. An Empirical Study of the Non-determinism of ChatGPT in Code Generation. arXiv:2308.02828.

Pendlebury, F.; Pierazzi, F.; Jordaney, R.; Kinder, J.; and Cavallaro, L. 2019. TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time. In *USENIX Security 2019*, 729–746.

Ratner, A.; Bach, S. H.; Ehrenberg, H.; Fries, J.; Wu, S.; and Ré, C. 2017. Snorkel: Rapid Training Data Creation with Weak Supervision. *Proceedings of the VLDB Endowment*, 11(3): 269–282.

Singh, R.; Tariq, S.; Jalalvand, F.; Chhetri, M. B.; Nepal, S.; Paris, C.; and Lochner, M. 2026. LLMs in the SOC: An Empirical Study of Human-AI Collaboration in Security Operations Centres. In *IEEE S&P*, 4262–4281.

Sommer, R.; and Paxson, V. 2010. Outside the Closed World: On Using Machine Learning for Network Intrusion Detection. In *IEEE S&P*, 305–316.

Tunstall, L.; Reimers, N.; Jo, U. E. S.; Bates, L.; Korat, D.; Wasserblat, M.; and Pereg, O. 2022. Efficient Few-Shot Learning Without Prompts. arXiv:2209.11055.

Turcotte, M.; Labrèche, F.; and Paquette, S.-O. 2025. Automated Alert Classification and Triage (AACT): An Intelligent System for the Prioritisation of Cybersecurity Alerts. arXiv:2505.09843.

TypeSafe AI. 2026. Jev: a decision model (System One). https://docs.typesafe.ai/models. Model `jev-1.14.0`, accessed 2026-11-01.

van Ede, T.; Aghakhani, H.; Spahn, N.; et al. 2022. DEEPCASE: Semi-Supervised Contextual Analysis of Security Events. In *IEEE S&P*, 522–539.

Wei, B.; Tay, Y. S.; Liu, H.; et al. 2025. CORTEX: Collaborative LLM Agents for High-Stakes Alert Triage. arXiv:2510.00311.

Xiong, M.; Hu, Z.; Lu, X.; et al. 2024. Can LLMs Express Their Uncertainty? An Empirical Evaluation of Confidence Elicitation in LLMs. In *ICLR*.

Yang, A.; Li, A.; Yang, B.; Zhang, B.; Hui, B.; et al. 2025. Qwen3 Technical Report. arXiv:2505.09388.

Yehudai, A.; and Bandel, E. 2024. FastFit: Fast and Effective Few-Shot Text Classification with a Multitude of Classes. In *NAACL 2024 Demos*, 174–184.